Connecting Board Table Of Conents.

Contents

COMPLETE LAB MANUAL CONTENT - COPY INTO WORD

Copy everything below into a new Word document:

BIP26 SENSATE-X LABORATORY MANUAL

Connecting TenStar ESP32-S3 to Waveshare LC29H GPS Module

NMEA Data Reception and Testing

Technological University Dublin
School of Electrical and Electronic Engineering

Course: BIP26 SENSATE-X Blended Intensive Programme
Academic Year: 2025/2026
Version: 1.0
Date: April 2026

Prepared by: Gerard Stockil, Senior Lecturer

TABLE OF CONTENTS

---

1. INTRODUCTION AND LEARNING OBJECTIVES

What You'll Build

In this laboratory session, you will connect a GPS receiver module (Waveshare LC29H) to a microcontroller (TenStar ESP32-S3) and receive authentic satellite navigation data. This is the foundation of the BIP26 project on Galileo OSNMA (Open Service Navigation Message Authentication).

Learning Objectives

By the end of this lab, you will be able to:

- Identify GPIO pins on the ESP32-S3 microcontroller
- Configure hardware jumpers correctly on the GPS module
- Connect UART serial devices using jumper wires
- Understand TX/RX signal direction
- Upload and run Arduino programs
- Receive and display NMEA sentences from GPS satellites
- Recognize valid GPS data
- Troubleshoot common connection problems

Time Required

Approximately 60-90 minutes for first-time setup and testing.

---

2. SAFETY INFORMATION

Electrical Safety

⚠️ **Important Safety Rules:**

- Always disconnect USB power before making or changing wire connections
- Never connect or disconnect wires while boards are powered
- Check all connections before applying power

- Do not short circuit any pins
- If you smell burning or see smoke, immediately disconnect power
- Work on a non-conductive surface (wood or plastic table)
- Keep liquids away from electronics

Component Handling

- Handle boards by the edges, avoid touching components
- Static electricity can damage electronic components - touch a grounded metal object before handling boards
- Do not force connectors - they should insert smoothly
- Store boards in anti-static bags when not in use

USB Power

- Use good quality USB cables rated for data transmission
- Do not exceed 5V power supply voltage
- Both boards operate at 3.3V logic levels - never connect to 5V GPIO pins
- USB ports on the same PC may share common ground (this is safe and correct)

---

3. HARDWARE REQUIRED

Main Components

Item	Specification	Quantity
TenStar ESP32-S3 Development Board	TS-ESP32-S3, USB-C	1
Waveshare LC29H GPS Module	LC29H(AA) with OSNMA, GPS HAT	1
Female-to-Male Jumper Wires	20cm length, various colors	3 minimum
USB-C Cable	Data-capable (not charge-only)	1
Micro-USB Cable	Data-capable	1
Personal Computer	Windows/macOS/Linux, 2 USB ports	1

Software Required

- Arduino IDE 2.x (installed and configured)
- ESP32 board support package (installed via Board Manager)

- CH340 USB driver (if not already installed)
- Serial Monitor (built into Arduino IDE)

Optional but Recommended

- Anti-static wrist strap
- Non-conductive work surface
- Multimeter (for advanced troubleshooting)
- Access to window or outdoors (for GPS signal reception)

---

4. UNDERSTANDING THE HARDWARE

TenStar ESP32-S3 Board

Key Features:

- ESP32-S3 dual-core processor (240 MHz)
- Wi-Fi and Bluetooth connectivity
- USB-C connector for power and programming
- Programmable via Arduino IDE
- 3.3V logic level (important!)

GPIO Pins We'll Use:

- **GPIO43:** TX (Transmit) - Sends data FROM ESP32 TO GPS
- **GPIO44:** RX (Receive) - Receives data FROM GPS TO ESP32
- **GND:** Ground reference (must be common between both boards)

USB-C Connector:

- Provides 5V power from PC
- Creates virtual COM port (COM3, COM4, etc.)
- Used for programming and Serial Monitor

Waveshare LC29H GPS Module

Key Features:

- Quectel LC29H(AA) GNSS chip
- Supports GPS, Galileo, GLONASS, BeiDou

- L1 + L5 dual-band reception
- OSNMA authentication capable (AA firmware)
- Integrated patch antenna (ceramic square on board)
- PPS (Pulse Per Second) LED output
- UART serial output at 9600 baud

UART Pins:

- **TX:** Transmits NMEA data FROM GPS TO microcontroller
- **RX:** Receives configuration commands FROM microcontroller TO GPS
- **GND:** Ground reference

Micro-USB Connector:

- Provides 5V power
- Can be used for direct USB-to-GPS communication (not used in this lab)

PPS LED:

- Blinks once per second when GPS has valid satellite fix
- Useful indicator that GPS is working

Patch Antenna:

- Built into the module (gray ceramic square)
- No external antenna needed
- Needs clear view of sky for best performance

---

5. WAVESHARE LC29H JUMPER CONFIGURATION

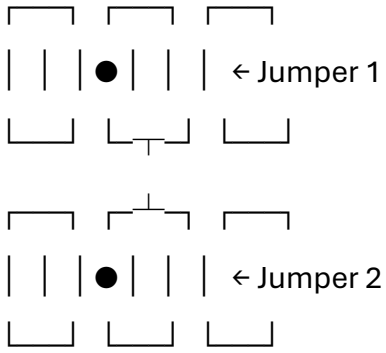
Understanding the Jumpers

The Waveshare board has two yellow jumpers that control the signal routing between:

- USB connector
- GPS chip (LC29H)
- External microcontroller (your ESP32)

Jumper Block Layout:

A B C



Position A: USB-LC29H (USB direct to GPS)

Position B: PI-LC29H (Microcontroller to GPS) ← CORRECT FOR ESP32

Position C: USB-PI (USB to microcontroller)

⚠️ **CRITICAL: Correct Jumper Settings**

For connection to ESP32, BOTH jumpers must be in position B:

✓ **Jumper 1:** Position B (PI-LC29H)
✓ **Jumper 2:** Position B (PI-LC29H)

This is the configuration shown in your reference photo.

What this means:

- ESP32 TX/RX pins connect directly to GPS chip
- UART communication enabled
- This is the standard configuration for BIP26

❌ **Wrong Settings (Do Not Use)**

Position A-A (USB-LC29H):

- Bypasses external connections
- GPS talks only to USB port
- ESP32 cannot receive data

Position C-C (USB-PI):

- Wrong signal routing
- Will not work for GPS connection

Position A-C or C-A (Mixed):

- Invalid configuration

- Undefined behavior

How to Set Jumpers

1. **Locate the yellow jumper block** (see reference photo)
2. **Check current position** of both jumpers
3. **If not in B-B position:**
    - Gently pull jumper straight up (do not twist)
    - Place jumper onto position B pins
    - Press down firmly but gently
    - Ensure metal contacts fully cover both pins
4. **Verify both jumpers are in position B**

[INSERT PHOTO: WaveShareJumper01.jpg showing B-B configuration]

5 A Connecting the Antennea of the Waveshrare board to the IPX Connector.

HOW TO CONNECT IPX/U.FL ANTENNA

What is an IPX Connector?

IPX (also called **U.FL** or **IPEX**) is a tiny snap-on RF connector commonly used for Wi-Fi and GPS antennas on development boards.

- Very small (about 2-3mm diameter)
- Snap-fit connection (press straight down)
- Fragile - handle carefully
- Rated for only 20-30 connection cycles

CONNECTION PROCEDURE:

⚠️ **IMPORTANT: Connect antenna BEFORE powering on the board (especially if using Wi-Fi)**

Step 1: Identify the Connector

- Look for a tiny gold-colored circular connector on the board
- Usually near the edge of the PCB

- May be labeled "ANT" or have antenna symbol

Step 2: Prepare the Antenna Cable

- The external antenna has a matching IPX/U.FL connector on one end
- Small metal snap connector (looks like a tiny button)
- Handle by the cable, not the connector

Step 3: Align the Connector

- Hold the board stable (don't flex the PCB)
- Position the antenna connector directly above the board connector
- **Alignment is critical** - connector must be perfectly centered

Step 4: Press Straight Down

- Use your fingernail or small flat tool
- Press **straight down** with firm, even pressure
- **Do NOT twist or angle** - only press vertically
- You should feel/hear a small "click" or "snap"

Step 5: Verify Connection

- Gently tug the antenna cable (not the connector)
- Should be firmly attached
- Connector should not wobble or lift off

⚠️ **CRITICAL WARNINGS:**

DO:

- ✓ Press straight down
- ✓ Use firm, even pressure
- ✓ Connect before powering on
- ✓ Support the PCB from underneath while pressing
- ✓ Handle by the cable, not the tiny connector

DON'T:

- ✗ Twist or rotate while connecting

- ✗ Pull at an angle when disconnecting
- ✗ Force if not aligned properly
- ✗ Touch the gold contacts (oils from fingers can degrade connection)
- ✗ Operate Wi-Fi/GPS without antenna connected (can damage RF circuitry)

DISCONNECTION (If Needed):

To remove antenna:

1. **Support the PCB** from underneath
2. **Grip the plastic housing** of the connector (not the cable)
3. **Pull straight up** with steady pressure
4. **Do NOT twist** - only vertical motion

Note: These connectors are rated for limited cycles (20-30). Avoid frequent connection/disconnection.

VIDEO REFERENCES:

Excellent tutorial videos for IPX/U.FL connection:

1. SparkFun - How to Connect U.FL/IPEX Connectors:
https://www.youtube.com/watch?v=d5A4xPXjjKQ

- Clear close-up demonstration
- Shows correct technique
- Short (2 minutes)

2. Adafruit - U.FL Antenna Cable Connection: https://www.youtube.com/watch?v=qyGZa0wC-To

- Good visual demonstration
- Shows common mistakes
- Explains alignment

3. Search terms for more videos:

- "IPX connector how to connect"
- "U.FL antenna connector tutorial"
- "IPEX antenna cable installation"

General YouTube search: "U.FL connector" or "IPX antenna connector"

ANTENNA PLACEMENT:

For Wi-Fi/Bluetooth (ESP32):

- Position antenna away from metal objects
- Keep antenna perpendicular to PCB if possible
- Don't coil or bend cable sharply
- Avoid placing antenna under metal enclosures

For GPS (if applicable):

- Antenna should have clear view of sky
- Position horizontally (flat)
- Away from other RF sources
- Near window or outdoors for best performance

TROUBLESHOOTING:

Problem: Connector won't snap on

- **Cause:** Misaligned
- **Solution:** Check alignment carefully, ensure centered before pressing

Problem: Connector falls off easily

- **Cause:** Not fully seated
- **Solution:** Reconnect with more pressure straight down

Problem: Connector damaged

- **Cause:** Twisted during connection/disconnection
- **Solution:** IPX connectors are fragile - may need replacement if damaged

Problem: Poor Wi-Fi/GPS performance

- **Cause:** Antenna not connected or poor connection
- **Solution:** Verify solid connection, check antenna placement

---

FOR BIP26 STUDENTS:

Simple instructions you could give them:

Connecting the External Antenna:

1. **Before powering on**, locate the small gold IPX connector on your board
2. **Align** the antenna connector directly above it (must be centered)
3. **Press straight down** firmly until you hear/feel a click
4. **Test:** Gently tug the cable - it should be firmly attached
5. **Position** the antenna away from metal objects

⚠️ **Important:** Press straight down only - do NOT twist!

Video tutorial: https://www.youtube.com/watch?v=d5A4xPXjjKQ

PHOTOS/DIAGRAMS TO INCLUDE:

If creating a document, include:

1. Photo of IPX connector location on board (close-up)
2. Photo of antenna cable connector end
3. Diagram showing alignment (top view)
4. Photo of properly connected antenna
5. Red X over twisted/angled connection (what NOT to do)

Would you like me to:

1. Create a separate one-page "Antenna Connection Guide" PDF-ready document?
2. Add this to the existing lab manual with photos?
3. Create a simple quick-reference card for students?

Also, please confirm:

- Is this for the **TenStar ESP32-S3** Wi-Fi antenna, the **Waveshare GPS** antenna, or both?
- Do both boards have IPX connectors with external antennas?

6. STEP-BY-STEP WIRING GUIDE

Wire Color Code for This Lab

We use a standard color coding system to minimize errors:

Wire Color	Connection Purpose	Signal Direction
Black	Ground (GND)	Common reference
Blue	ESP32 TX → GPS RX	ESP32 sends to GPS
Yellow	GPS TX → ESP32 RX	GPS sends to ESP32

Important: TX (Transmit) always connects to RX (Receive) and vice versa. This is because one device's output is the other device's input.

Connection Table

TenStar ESP32-S3	Wire Color	Waveshare LC29H GPS
GND	Black	GND
GPIO43 (TX)	Blue	RX
GPIO44 (RX)	Yellow	TX

Step-by-Step Connection Procedure

⚠️ **CRITICAL: Both boards MUST be disconnected from USB power during wiring!**

Step 1: Prepare Your Workspace

1. Clear your work area of metal objects and liquids
2. Place both boards on non-conductive surface
3. Ensure both boards are **unpowered** (no USB cables connected)
4. Lay out your three jumper wires (black, blue, yellow)
5. Touch a grounded metal object to discharge static

Step 2: Verify Jumper Settings

1. Check Waveshare jumpers are in **B-B position** (see Section 5)
2. If not, adjust them now before making any connections

Step 3: Connect Ground (Black Wire)

1. **Identify GND pins:**
    - TenStar: GND pin (labeled GND on board)
    - Waveshare: GND pin (labeled GND)
2. **Insert black wire:**
    - Female end → TenStar GND pin
    - Male end → Waveshare GND pin
3. **Verify:** Wire is firmly seated, no wobble

Why ground first? Ground provides the voltage reference for all signals. It must be connected before any data lines.

Step 4: Connect ESP32 TX to GPS RX (Blue Wire)

1. **Identify TX/RX pins:**
    - TenStar: GPIO43 (this is the TX pin)
    - Waveshare: RX pin (labeled RX)
2. **Insert blue wire:**
    - Female end → TenStar GPIO43
    - Male end → Waveshare RX
3. **Verify:** Connection is secure

Remember: ESP32 TX sends data TO GPS RX

Step 5: Connect GPS TX to ESP32 RX (Yellow Wire)

1. **Identify TX/RX pins:**
    - Waveshare: TX pin (labeled TX)
    - TenStar: GPIO44 (this is the RX pin)
2. **Insert yellow wire:**
    - Male end → Waveshare TX
    - Female end → TenStar GPIO44
3. **Verify:** Connection is secure

Remember: GPS TX sends data TO ESP32 RX

Step 6: Visual Inspection

Before applying power, carefully check:

- [ ] Black wire: GND to GND ✓
- [ ] Blue wire: ESP32 GPIO43 (TX) to GPS RX ✓
- [ ] Yellow wire: GPS TX to ESP32 GPIO44 (RX) ✓
- [ ] Jumpers on Waveshare in B-B position ✓
- [ ] No wires touching each other (no shorts) ✓
- [ ] All wires firmly inserted ✓
- [ ] Boards on non-conductive surface ✓

[INSERT PHOTOS: TenStartWaveshareConn01.jpg, TWConnect03.jpg, TWConnection02.jpg showing correct connections]

---

7. PRE-CONNECTION SAFETY CHECKS

Before connecting USB power, complete this checklist:

Final Safety Checklist

Check	Status
1. Black wire connected: GND to GND	[ ]
2. Blue wire connected: ESP32 TX (GPIO43) to GPS RX	[ ]
3. Yellow wire connected: GPS TX to ESP32 RX (GPIO44)	[ ]
4. Waveshare jumpers in B-B position	[ ]
5. No wire shorts (wires not touching each other)	[ ]
6. Boards on non-conductive surface	[ ]
7. Workspace clear of liquids and metal objects	[ ]
8. Good quality USB cables ready	[ ]
9. Arduino IDE open and ready	[ ]
10. You have read and understood all instructions	[ ]

⚠️ **Do not proceed to power-on until ALL boxes are checked!**

7A. POWER-UP SAFETY AND SEQUENCE

Is Partial Powering Safe?

Yes! If you forget to power one board, no damage will occur:

✓ **GPS powered, ESP32 not:** Safe - ESP32 inputs are protected
✓ **ESP32 powered, GPS not:** Safe - GPS inputs are protected
✓ **Both powered simultaneously:** Safe - normal operation

Why it's safe:

- Both boards use 3.3V logic levels (no voltage mismatch)
- Common ground connection via black wire provides reference
- Built-in ESD protection diodes on all GPIO pins
- Current-limited outputs prevent damage from shorts
- Modern circuit design with robust protection

Recommended Power-Up Sequence

For best results and clearest diagnostics, follow this sequence:

Step 1: Verify Connections (Both Boards Unpowered)

- Check all three wires are connected correctly
- Black wire: GND to GND
- Blue wire: ESP32 GPIO43 (TX) to GPS RX
- Yellow wire: GPS TX to ESP32 GPIO44 (RX)
- Verify jumpers on GPS in B-B position (PI-LC29H mode)

Step 2: Power GPS Module First

1. Connect Waveshare Micro-USB to PC
2. Wait 5 seconds for GPS to initialize
3. Check power LED is lit (usually red or green)
4. Wait 30-60 seconds - PPS LED should start blinking once per second
5. PPS blinking indicates GPS has acquired satellite fix

Step 3: Power ESP32 Module Second

1. Connect TenStar USB-C to PC

2. Wait 5 seconds for board to boot
3. Check power LED is lit
4. In Arduino IDE: Tools → Port → Select COM4 (or your ESP32 port)
5. Upload program or open Serial Monitor

Why this sequence is recommended:

- GPS begins satellite acquisition immediately (can take 30-60 seconds)
- By the time ESP32 boots, GPS may already have satellite fix
- ESP32 receives valid NMEA data immediately when program starts
- Avoids potential back-powering of GPS through signal pins
- Easier diagnostics: you can verify GPS works independently first
- PPS LED blinking confirms GPS is functional before connecting ESP32

Alternative: Simultaneous Power-Up

Can I plug both in at the same time?

Yes, absolutely! Most students will connect both USB cables simultaneously, and this is perfectly safe. The recommended sequence above is for optimal results and diagnostics, not for safety reasons.

What happens with simultaneous power-up:

- Both boards power up at roughly the same time
- USB ports provide stable power to each board
- Common ground (black wire) prevents voltage differences
- Both boards initialize normally
- This is completely safe and works fine

Power-Down Sequence

Recommended (but not critical) sequence:

Step 1: Disconnect ESP32 First

- Unplug USB-C cable from ESP32
- This stops the program from running

Step 2: Disconnect GPS Second

- Unplug Micro-USB from GPS

- GPS powers down

Note: Power-down order is not critical for safety. You can disconnect in any order or simultaneously without causing damage.

⚠️ **Important Safety Rules**

❌ **DO NOT:**

- Connect or disconnect jumper wires while boards are powered
- Force connectors - they should insert smoothly
- Allow wires to touch adjacent pins or each other
- Touch components or pins while powered
- Work with wet hands or near liquids
- Use damaged USB cables

✅ **ALWAYS:**

- Power off both boards before making any wiring changes
- Double-check all connections before applying power
- Verify jumper settings before power-on
- Keep work area clear of metal objects and liquids
- Use good quality, data-capable USB cables
- Work on non-conductive surface

What If I Forget to Power One Board?

Don't worry - nothing will be damaged!

Scenario A: GPS is unpowered (forgot Micro-USB)

What you'll see:

- ESP32 powers up normally
- Serial Monitor shows startup banner
- "Waiting for NMEA data from GPS..." appears
- No NMEA sentences appear (GPS not sending data)

Solution:

- Simply connect GPS Micro-USB cable
- No need to disconnect or restart ESP32

- NMEA data will begin appearing immediately

Scenario B: ESP32 is unpowered (forgot USB-C)

What you'll see:

- GPS operates normally
- PPS LED may blink (if GPS has satellite fix)
- No Serial Monitor output (ESP32 not running)
- Arduino IDE shows "Port not available" or similar

Solution:

- Simply connect ESP32 USB-C cable
- Arduino IDE will detect the port
- Select port and upload program
- Or open Serial Monitor if program already uploaded

Important: In both scenarios, you can safely connect the missing power cable without disconnecting anything. The design protects against partial power-up.

Protection Mechanisms Explained

Why is partial powering safe?

1. Common Ground Reference

- Black wire connects both boards' ground pins
- Provides stable voltage reference for both boards
- Prevents dangerous voltage differences
- Return path for protection diode currents

2. Matching Voltage Levels

- ESP32 GPIO: 3.3V logic levels
- GPS UART: 3.3V logic levels
- No voltage level shifters needed
- No risk of overvoltage damage

3. ESD Protection Diodes

- Every GPIO pin has built-in protection diodes
- Diodes clamp voltage to safe range (GND to VCC + 0.6V)

- If unpowered pin receives signal, diodes safely conduct small current
- Prevents damage from signals on unpowered inputs

4. Current-Limited Outputs

- ESP32 GPIO pins limited to ~20-40mA output current
- GPS UART outputs similarly current-limited
- Even if shorted, current is too low to cause damage
- Built-in overcurrent protection shuts down output if needed

5. Separate Power Domains

- Each board powered from separate USB port
- No shared power rails that could cause issues
- Isolation between power supplies
- Each board has its own voltage regulation

Common Mistakes That Are Still Safe

Mistake 1: Reversed TX/RX Wires

What happens:

- Blue wire: ESP32 TX (GPIO43) → GPS TX (both outputs connected)
- Yellow wire: ESP32 RX (GPIO44) → GPS RX (both inputs connected)

Safe? ✅ Yes - won't damage anything
Works? ❌ No - no data transfer occurs

Why safe:

- Two outputs driving each other: current-limited, protected
- Two inputs connected: just float to same voltage
- No damage, just no communication

Solution: Power off, swap blue and yellow wires

Mistake 2: Missing Ground Wire

What happens:

- Only blue and yellow wires connected
- Black ground wire forgotten

Safe? ⚠️ Potentially risky - may work poorly or damage from ESD

Works? ❌ Usually not - no common ground reference

Why risky:

- No stable voltage reference between boards
- Floating ground can cause unpredictable behavior
- Increased susceptibility to static discharge
- May partially work due to ground through USB PC connection

Solution: Power off, connect black ground wire

Mistake 3: Wires Touch During Connection

What happens:

- Adjacent wires briefly short together
- Momentary short circuit

Safe? ✅ Yes - GPIO pins are current-limited

Works? ✅ Yes - once properly connected

Why safe:

- Brief shorts don't carry enough current to damage
- Protection circuits activate
- Modern boards designed to handle this

Best practice: Still avoid by powering off before wiring

What Can Actually Cause Damage?

Very few things can damage these boards:

❌ **Applying voltage >3.6V to GPIO pins**

- Don't connect 5V signals directly to ESP32 or GPS
- Both boards use 3.3V logic only
- Our setup is all 3.3V, so this won't happen

❌ **Sustained short circuit to power rails**

- Shorting 3.3V to GND with wire for extended time
- USB port will likely shut down first (protection)
- Don't do this intentionally!

❌ **Static discharge without grounding**

- Touch board with static charge built up
- Can damage sensitive components
- Prevention: touch grounded metal before handling boards

❌ **Liquid spills**

- Water, coffee, etc. on powered electronics
- Can cause shorts and damage
- Keep liquids away from work area

❌ **Excessive physical force**

- Breaking connectors, bending pins
- Forcing components that don't fit
- Handle boards gently by edges

Note: Normal connection mistakes (wrong wiring, partial power) will NOT cause damage.

Power-Up Troubleshooting

Problem: Neither board powers up when connected

Possible causes:

- USB ports not providing power (PC in sleep mode)
- Damaged USB cables
- USB hub not powered

Solution:

- Try different USB ports directly on PC
- Use known-good USB cables
- Ensure PC is awake and running

Problem: One board powers up, other doesn't

Possible causes:

- Loose USB connection
- Damaged USB cable
- Faulty USB port

Solution:

- Check USB cable firmly seated
- Try different USB cable
- Try different USB port on PC

Problem: Boards power up but get very hot

Possible causes:

- Short circuit somewhere
- Damaged component

Solution:

- **Immediately disconnect power**
- Inspect for shorts (wires touching, solder bridges)
- Check for damaged components
- If unclear, ask instructor for help

Problem: PC doesn't recognize ESP32 (no COM port)

Possible causes:

- USB cable is charge-only (no data lines)
- CH340 driver not installed
- Damaged USB connector

Solution:

- Try different USB cable (must be data-capable)
- Install CH340 driver (see earlier instructions)
- Try different USB port

SAFETY SUMMARY TABLE

Scenario	GPS Power	ESP32 Power	Safe?	Works?	Notes
Normal operation	✓ On	✓ On	✅ Yes	✅ Yes	Recommended configuration

Scenario	GPS Power	ESP32 Power	Safe?	Works?	Notes
GPS only powered	✓ On	✗ Off	✅ Yes	❌ No	GPS works independently; ESP32 idle
ESP32 only powered	✗ Off	✓ On	✅ Yes	❌ No	ESP32 runs but no GPS data received
Both unpowered	✗ Off	✗ Off	✅ Yes	❌ No	Safe for wiring connections
GPS first (recommended)	✓ On first	✓ On after	✅ Yes	✅ Yes	Best practice for diagnostics
Both simultaneous	✓ Together	✓ Together	✅ Yes	✅ Yes	Also perfectly fine
Reversed TX/RX	✓ On	✓ On	✅ Yes	❌ No	No damage; no communication
Missing GND wire	✓ On	✓ On	⚠️ Risky	❌ No	May damage from ESD; likely won't work
Shorted wires	✓ On	✓ On	✅ Yes*	❌ No	*Brief shorts OK; sustained shorts may damage

---

This section is now complete and ready to add to your lab manual!

Insert this section between:

- **After:** Section 7 (Pre-Connection Safety Checks)
- **Before:** Section 8 (Arduino IDE Setup)

The power-up safety information is now comprehensive and will help students understand that the design is robust and forgiving of common mistakes, while still encouraging best practices.

Is there anything else you'd like me to add or clarify in the lab manual?

8. ARDUINO IDE SETUP

Before Uploading Code

Ensure Arduino IDE is properly configured:

Step 1: Open Arduino IDE

- Launch Arduino IDE 2.x
- Run as administrator (Windows: right-click → Run as administrator)

Step 2: Select Board

- **Tools → Board → esp32 → ESP32S3 Dev Module**

Step 3: Select Port

- Keep both boards unplugged for now
- We'll select the port after power-on

Step 4: Verify Settings

Check these settings under **Tools** menu:

Board: "ESP32S3 Dev Module"

Upload Speed: "921600"

USB Mode: "Hardware CDC and JTAG"

USB CDC On Boot: "Enabled"

USB Firmware MSC On Boot: "Disabled"

USB DFU On Boot: "Disabled"

Upload Mode: "UART0 / Hardware CDC"

CPU Frequency: "240MHz (WiFi)"

Flash Mode: "QIO 80MHz"

Flash Size: "16MB (128Mb)"

Partition Scheme: "Default 4MB with spiffs"

Core Debug Level: "None"

PSRAM: "QSPI PSRAM"

Arduino Runs On: "Core 1"

Events Run On: "Core 1"

Erase All Flash Before Sketch Upload: "Disabled"

Port: (will select after power-on)

9. TEST PROGRAM - SIMPLE NMEA DISPLAY

Program Purpose

This program receives raw NMEA sentences from the GPS module and displays them on your computer screen via the Serial Monitor.

Hwew is a minimal verion with few comments

/* * BIP26 GPS NMEA Reader * GPS TX→GPIO44, GPS RX→GPIO43, GND→GND, Jumpers: B-B */ void setup() { Serial.begin(115200); // USB to computer delay(1000); Serial.println("BIP26 GPS NMEA Reader\nWaiting for GPS data...\n"); Serial1.begin(9600, SERIAL_8N1, 44, 43); // GPS: 9600 baud, RX=44, TX=43 delay(2000); } void loop() { if (Serial1.available()) { // Check for GPS data String nmea = Serial1.readStringUntil('\n'); // Read NMEA sentence nmea.trim(); // Remove whitespace Serial.println(nmea); // Display } }

What it does:

1. Initializes Serial (USB to computer) at 115200 baud
2. Initializes Serial1 (UART to GPS) at 9600 baud on GPIO43/44
3. Reads complete NMEA sentences from GPS
4. Displays sentences in Serial Monitor
5. Runs continuously

Complete Test Program

Copy this code into Arduino IDE:

```
/*
 * ========================================
 * BIP26 SENSATE-X - GPS NMEA Test Program
 * ========================================
 *
 * Hardware: TenStar ESP32-S3 + Waveshare LC29H GPS
 * Purpose: Display raw NMEA sentences from GPS
 *
 * HARDWARE CONNECTIONS:
 * ---------------------
 * GPS TX (yellow wire) → ESP32 GPIO44 (RX)
 * GPS RX (blue wire)   → ESP32 GPIO43 (TX)
```

```
 * GPS GND (black wire) → ESP32 GND
 *
 * WAVESHARE JUMPERS:
 * ------------------
 * Both jumpers in position B (PI-LC29H mode)
 *
 * POWER:
 * ------
 * ESP32: USB-C to PC
 * GPS: Micro-USB to PC (separate port)
 *
 * AUTHOR: BIP26 Team, TU Dublin
 * DATE: April 2026
 * VERSION: 1.0
 */

// ========================
// NO LIBRARIES REQUIRED
// ========================
// This simple test uses only built-in Arduino functions

// ========================
// SETUP - RUNS ONCE
// ========================
void setup() {

  // ----------------------------------------
  // Initialize USB Serial (to computer)
  // ----------------------------------------
  // This is the Serial Monitor connection
  // Baud rate: 115200 (standard for ESP32)
  Serial.begin(115200);

  // Wait for Serial Monitor to connect
  delay(1000);

  // Print startup banner
  Serial.println("=========================================");
  Serial.println("BIP26 GPS NMEA Reader - Test Program");
  Serial.println("TenStar ESP32-S3 + Waveshare LC29H");
  Serial.println("=========================================");
  Serial.println();

  // ----------------------------------------
  // Initialize Serial1 (to GPS module)
  // ----------------------------------------
  // Serial1.begin(baud, format, RX_pin, TX_pin)
  //
```

```
  // baud: 9600 (GPS standard)
  // format: SERIAL_8N1 (8 data bits, No parity, 1 stop bit)
  // RX_pin: GPIO44 (receives FROM GPS)
  // TX_pin: GPIO43 (sends TO GPS)
  Serial1.begin(9600, SERIAL_8N1, 44, 43);

  // Print configuration info
  Serial.println("Serial1 Configuration:");
  Serial.println("  Baud rate: 9600 bits/second");
  Serial.println("  Format: 8N1 (8 data bits, No parity, 1 stop bit)");
  Serial.println("  RX pin: GPIO44 (receives from GPS TX)");
  Serial.println("  TX pin: GPIO43 (sends to GPS RX)");
  Serial.println();

  Serial.println("Waiting for NMEA data from GPS...");
  Serial.println("Note: GPS needs clear view of sky for satellite fix");
  Serial.println("     PPS LED on GPS should blink once per second");
  Serial.println();

  // Brief delay to let GPS module stabilize
  delay(2000);

  Serial.println("--- NMEA DATA STREAM (Real-time) ---");
  Serial.println();
}

// ========================
// LOOP - RUNS FOREVER
// ========================
void loop() {

  // ------------------------------------------
  // Check if GPS has sent any data
  // ------------------------------------------
  // Serial1.available() returns the number of bytes
  // waiting to be read from GPS module
  if (Serial1.available()) {

    // ------------------------------------------
    // Read one complete NMEA sentence
    // ------------------------------------------
    // NMEA sentences end with \n (newline)
    // readStringUntil() waits until \n is received
    // then returns everything before the \n
    String nmeaSentence = Serial1.readStringUntil('\n');

    // ------------------------------------------
    // Clean up the string
```

```
  // ----------------------------------------
  // Remove any trailing whitespace, \r, or \n characters
  nmeaSentence.trim();

  // ----------------------------------------
  // Display on Serial Monitor
  // ----------------------------------------
  // Print the complete NMEA sentence
  // This goes to your computer screen via USB
  Serial.println(nmeaSentence);

  // How this works:
  // 1. GPS transmits data at 9600 baud (9600 bits/second)
  // 2. ESP32 Serial1 receives data on GPIO44
  // 3. Data is stored in Serial1 buffer
  // 4. We read one line at a time (until \n)
  // 5. We send it to Serial (115200 baud to computer)
  // 6. Serial Monitor displays it on screen
 }

 // ----------------------------------------
 // Loop continues immediately
 // ----------------------------------------
 // No delay() here - we want to catch GPS data
 // as soon as it arrives
 // Loop runs thousands of times per second
 // Serial1.available() returns 0 when no data waiting
}

/*
 * ========================================
 * EXPECTED OUTPUT IN SERIAL MONITOR
 * ========================================
 *
 * You should see lines like this scrolling continuously:
 *
 * $GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47
 * $GPGSA,A,3,04,05,09,12,,,,,,,,,2.5,1.3,2.1*39
 * $GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,003.1,W*6A
 * $GPGSV,3,1,11,03,03,111,00,04,15,270,00,06,01,010,00,13,06,292,00*74
 * $GPGSV,3,2,11,14,25,170,00,16,57,208,39,18,33,052,00,19,01,310,00*7D
 * $GPGSV,3,3,11,22,42,067,42,24,14,311,43,27,05,244,00*4D
 * $GPGLL,4807.038,N,01131.000,E,123519,A*2D
 *
 * Each line is one NMEA sentence
 * All start with $ and contain comma-separated data
 * All end with *XX (checksum in hexadecimal)
 *
```

```
* Different sentence types:
* - $GPGGA: Global Positioning System Fix Data
* - $GPRMC: Recommended Minimum Specific GPS Data
* - $GPGSV: GPS Satellites in View
* - $GPGSA: GPS DOP and Active Satellites
* - $GPGLL: Geographic Position - Latitude/Longitude
*
* Data updates approximately once per second
*
* ========================================
* SUCCESS INDICATORS
* ========================================
*
* ✓ Serial Monitor shows scrolling NMEA sentences
* ✓ Lines start with $GP or $GA or $GL
* ✓ Lines end with *XX (two hex digits)
* ✓ PPS LED on GPS blinks once per second
* ✓ New data appears continuously
*
* If you see this: YOUR CONNECTION IS WORKING PERFECTLY!
*
* ========================================
* TROUBLESHOOTING
* ========================================
*
* PROBLEM: Nothing appears in Serial Monitor
* SOLUTION:
* - Check wire connections (black, blue, yellow)
* - Verify Serial Monitor baud rate is 115200
* - Check correct COM port selected in Arduino IDE
* - Ensure both boards are powered
*
* PROBLEM: Gibberish characters (random symbols)
* SOLUTION:
* - Wrong baud rate on Serial1 (should be 9600)
* - Reupload program
* - Check Serial Monitor is set to 115200
*
* PROBLEM: "Waiting for NMEA data..." never changes
* SOLUTION:
* - Check Waveshare jumpers in B-B position
* - Verify TX/RX not swapped (blue to RX, yellow to TX)
* - Check GND connection (black wire)
* - Ensure GPS module is powered (Micro-USB connected)
*
* PROBLEM: Very few sentences or irregular updates
* SOLUTION:
* - GPS needs clear view of sky
```

```
* - Move near window or go outdoors
* - Wait 30-60 seconds for GPS to acquire satellites
* - Check PPS LED - should blink once per second
*
* PROBLEM: All sentences show invalid data (empty fields)
* SOLUTION:
* - GPS doesn't have satellite fix yet
* - Move to location with better sky visibility
* - Wait longer (cold start can take 30-60 seconds)
* - Check antenna is not obstructed
*/
```

---

10. POWER-ON AND TESTING PROCEDURE

Step-by-Step Power-On Sequence

Follow this exact sequence to minimize problems:

Step 1: Final Pre-Power Check

- Verify all wire connections one last time
- Check jumpers are in B-B position
- Ensure Arduino IDE is open with test program ready

Step 2: Connect GPS Module First

1. **Plug Waveshare LC29H into PC via Micro-USB**
2. **Observe:**
    - Power LED should illuminate (red or green)
    - After 30-60 seconds, PPS LED should blink once per second
    - PPS blinking means GPS has satellite fix
3. **If PPS doesn't blink:** Module needs clearer view of sky (move near window)

Step 3: Connect ESP32 Module

1. **Plug TenStar ESP32-S3 into PC via USB-C**
2. **Observe:**
    - Board power LED should illuminate
    - Windows may make USB connection sound
3. **Wait 5 seconds** for board to fully initialize

Step 4: Select COM Port in Arduino IDE

1. **Tools → Port**
2. **Look for:** "COM4 (ESP32S3 Dev Module)" or similar
3. **Select the ESP32 port** (probably COM4 based on your earlier testing)
4. **Note:** The GPS module may also create a COM port - ignore it, use the ESP32 port only

Step 5: Upload Test Program

1. **Click Upload button** (→ icon) in Arduino IDE
2. **Wait for upload to complete** (should take 10-30 seconds)
3. **Look for:** "Hard resetting via RTS pin..." at end of upload
4. **If upload fails:** Try holding BOOT button, pressing RESET, releasing BOOT during upload

Step 6: Open Serial Monitor

1. **Tools → Serial Monitor** (or Ctrl+Shift+M)
2. **Set baud rate:** 115200 (bottom right of Serial Monitor window)
3. **Set line ending:** "Both NL & CR" or "Newline" (bottom right dropdown)

Step 7: Observe Output

You should immediately see:

```
=====================================
BIP26 GPS NMEA Reader - Test Program
TenStar ESP32-S3 + Waveshare LC29H
=====================================

Serial1 Configuration:
  Baud rate: 9600 bits/second
  Format: 8N1 (8 data bits, No parity, 1 stop bit)
  RX pin: GPIO44 (receives from GPS TX)
  TX pin: GPIO43 (sends to GPS RX)

Waiting for NMEA data from GPS...
Note: GPS needs clear view of sky for satellite fix
    PPS LED on GPS should blink once per second

--- NMEA DATA STREAM (Real-time) ---

$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47
$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,003.1,W*6A
[...more NMEA sentences scrolling...]
```

Step 8: Verify Success

✓ **Startup banner appears**
✓ **NMEA sentences scroll continuously**
✓ **Lines start with $GP, $GA, or $GL**
✓ **Lines end with XX (two hex digits)*
✓ **PPS LED on GPS blinks once per second**
✓ **New data appears every second**

🎉 **If you see all of the above: SUCCESS! Your connection is working perfectly!**

---

11. EXPECTED OUTPUT AND VERIFICATION

What You Should See

When everything is working correctly, your Serial Monitor will display a continuous stream of NMEA sentences similar to this:

$GPGGA,095107.000,5313.7768,N,00619.4456,W,1,08,1.0,35.2,M,53.0,M,,*5E

$GPGSA,A,3,07,02,26,27,09,04,15,,,,,,1.7,1.0,1.3*35

$GPRMC,095107.000,A,5313.7768,N,00619.4456,W,0.03,165.48,020426,,,A*7E

$GPVTG,165.48,T,,M,0.03,N,0.06,K,A*37

$GPGGA,095108.000,5313.7768,N,00619.4456,W,1,08,1.0,35.2,M,53.0,M,,*51

$GPGSA,A,3,07,02,26,27,09,04,15,,,,,,1.7,1.0,1.3*35

$GPRMC,095108.000,A,5313.7768,N,00619.4456,W,0.03,165.48,020426,,,A*71

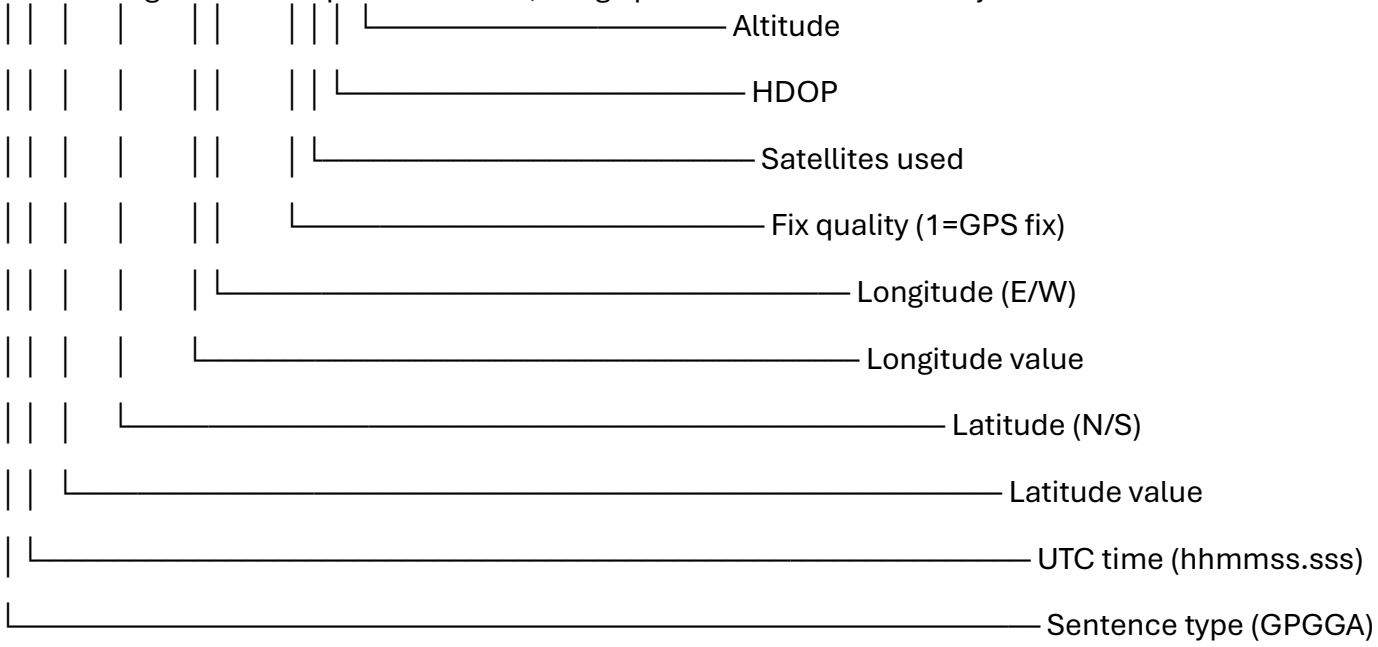
Understanding What You're Seeing

Sentence Structure:

Every NMEA sentence follows this format:

$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47

```
||| |   |   ||   |||| |  || ||||
||| |   |   ||   |||| |  || |└ Checksum
||| |   |   ||   |||| |  || └── Empty field
||| |   |   ||   |||| |  |└ ─── Empty field
||| |   |   ||   |||| |  └──── M (meters)
||| |   |   ||   |||| └─────── Geoid separation
||| |   |   ||   |||└ ──────── M (meters)
```



Common Sentence Types:

Sentence	Full Name	Contains
$GPGGA	Global Positioning System Fix Data	Time, position, fix quality, altitude
$GPRMC	Recommended Minimum Specific GPS Data	Time, date, position, speed, course
$GPGSA	GPS DOP and Active Satellites	Which satellites are being used, accuracy
$GPGSV	GPS Satellites in View	Detailed info about visible satellites
$GPGLL	Geographic Position - Lat/Long	Simple position data
$GPVTG	Track Made Good and Ground Speed	Speed and heading

Note: You may also see $GA... (Galileo), $GL... (GLONASS), or $GB... (BeiDou) sentences depending on which satellite systems are configured.

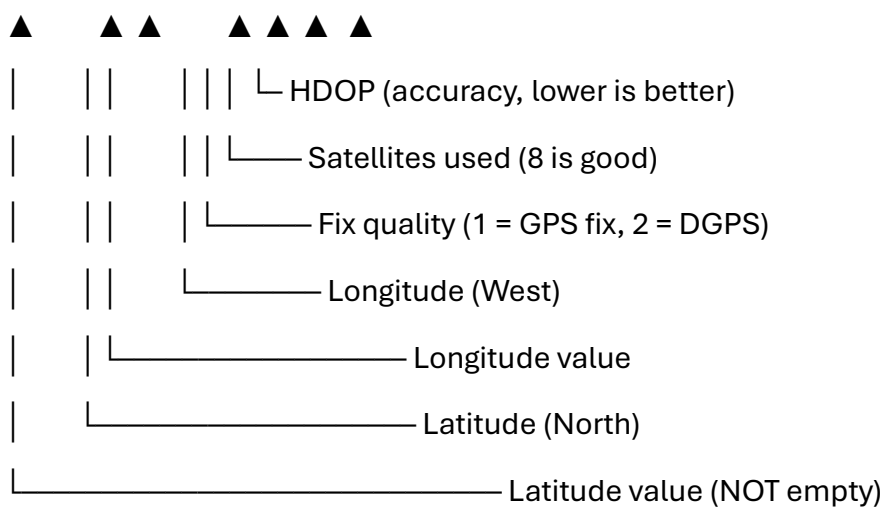
Success Indicators

✓ **Sentences appear continuously** (not just once)
✓ **Multiple sentence types** ($GPGGA, $GPRMC, $GPGSV, etc.)
✓ **Comma-separated data** (not random characters)
✓ **Checksum present** (*XX at end of each line)
✓ **Data updates** (time field changes every second)
✓ **PPS LED blinking** on GPS module

What Good GPS Fix Data Looks Like

When GPS has a **good fix** (locked onto satellites):

$GPGGA,095107.000,5313.7768,N,00619.4456,W,1,08,1.0,35.2,M,53.0,M,,*5E



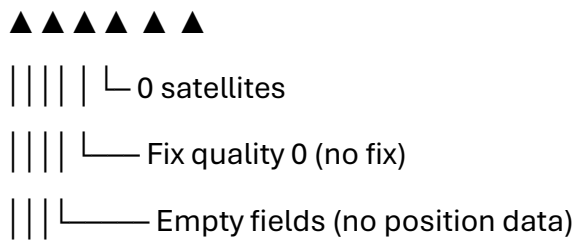
Good fix characteristics:

- Latitude and longitude fields have values (not empty)
- Fix quality is 1 or 2 (not 0)
- 4+ satellites being used
- HDOP less than 3.0 (lower is better accuracy)

What Bad/No Fix Data Looks Like

When GPS does **NOT** have a fix:

$GPGGA,095107.000,,,,,0,00,,,M,,M,,*XX



No fix characteristics:

- Latitude/longitude fields are empty
- Fix quality is 0
- 0 or very few satellites
- This is normal immediately after power-on

If you see this: GPS is working but hasn't locked onto satellites yet. Wait 30-60 seconds near a window or outdoors.

12. UNDERSTANDING NMEA SENTENCES

NMEA 0183 Protocol

NMEA stands for **National Marine Electronics Association**. NMEA 0183 is a standard for communication between marine electronic devices, now widely used in GPS receivers.

Key characteristics:

- ASCII text format (human-readable)
- Comma-separated values
- Each sentence ends with checksum
- Update rate typically 1 Hz (once per second)
- Transmitted serially at 9600 baud (standard)

Sentence Structure Explained

General format:

$AACCC,data,data,data,...,data*CS<CR><LF>

```
||||              ||  | |
||||              ||  | └─ Line Feed (\n)
||||              ||  └─── Carriage Return (\r)
||||              |└─────── Checksum (2 hex digits)
||||              └──────── Checksum delimiter (*)
|||└──────────────────────── Sentence type (3 characters)
||└──────────────────────── Talker ID (2 characters)
|└──────────────────────── Start character ($)
```

Talker IDs:

- **GP** = GPS (US)
- **GA** = Galileo (EU)
- **GL** = GLONASS (Russia)
- **GB** = BeiDou (China)
- **GN** = Combined (multiple systems)

Detailed Sentence Examples

$GPGGA - Global Positioning System Fix Data

Connecting Boards 5 April 2025 Ver A, Large parts AI Genearated. Subject to Revision.
Most important sentence for basic positioning.

$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47

Field breakdown:

1. 123519 = UTC time (12:35:19)

2. 4807.038 = Latitude 48°07.038' (ddmm.mmm format)

3. N = North

4. 01131.000 = Longitude 011°31.000' (dddmm.mmm format)

5. E = East

6. 1 = Fix quality (0=invalid, 1=GPS fix, 2=DGPS fix)

7. 08 = Number of satellites being tracked

8. 0.9 = Horizontal dilution of precision (HDOP)

9. 545.4 = Altitude above mean sea level

10. M = Units (meters)

11. 46.9 = Height of geoid above WGS84 ellipsoid

12. M = Units (meters)

13. (empty) = Time since last DGPS update

14. (empty) = DGPS station ID

*47 = Checksum

$GPRMC - Recommended Minimum Specific GPS Data

Contains date, time, position, speed, and course.

$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,003.1,W*6A

Field breakdown:

1. 123519 = UTC time (12:35:19)

2. A = Status (A=active/valid, V=void/invalid)

3. 4807.038 = Latitude 48°07.038'

4. N = North

5. 01131.000= Longitude 011°31.000'

6. E = East

7. 022.4 = Speed over ground in knots

8. 084.4 = Track angle in degrees

9. 230394 = Date (23 March 1994)

10. 003.1 = Magnetic variation in degrees

11. W = West

*6A = Checksum

$GPGSV - GPS Satellites in View

Shows which satellites are visible and their signal strength.

$GPGSV,3,1,11,03,03,111,00,04,15,270,00,06,01,010,00,13,06,292,00*74

Field breakdown:

1. 3 = Total number of GSV messages (this is message 1 of 3)

2. 1 = Message number

3. 11 = Total number of satellites in view

4-7. Satellite data block 1:

03 = Satellite PRN number

03 = Elevation in degrees

111 = Azimuth in degrees

00 = SNR (signal-to-noise ratio) in dB (00 = not tracking)

8-11. Satellite data block 2

12-15. Satellite data block 3

16-19. Satellite data block 4

*74 = Checksum

Checksum Calculation

The checksum is an XOR of all characters between $ and *.

Example: For $GPGGA,123519,4807.038,N*47

1. Take characters: GPGGA,123519,4807.038,N
2. XOR all ASCII values
3. Result in hexadecimal: 47

The receiver can verify data integrity by recalculating and comparing checksums.

---

13. TROUBLESHOOTING GUIDE

Problem: No Output in Serial Monitor

Symptoms:

- Serial Monitor is blank
- No startup banner appears
- No NMEA sentences

Solutions:

1. **Check Serial Monitor baud rate**
    - Must be set to **115200**
    - Check bottom-right corner of Serial Monitor window
2. **Verify correct COM port selected**
    - Tools → Port → Should show COM4 (or your ESP32 port)
    - Unplug/replug ESP32 to see which port appears/disappears
3. **Check USB cable**
    - Must be data-capable cable (not charge-only)
    - Try different USB cable
    - Try different USB port on PC
4. **Reupload program**
    - Click Upload button again
    - Wait for "Hard resetting via RTS pin..."
    - Open Serial Monitor again
5. **Check board power**
    - ESP32 power LED should be lit

- Try pressing RESET button on ESP32

Problem: Gibberish Characters

Symptoms:

- Random symbols: â•šÑ╗╗ ╟Ã©
- Unreadable text
- Not proper NMEA format

Solutions:

1. **Wrong Serial Monitor baud rate**
   - Set to **115200** (not 9600)
2. **Wrong Serial1 baud rate in code**
   - Should be Serial1.begin(9600, SERIAL_8N1, 44, 43)
   - Not 115200 or other values
3. **Verify code uploaded correctly**
   - Reupload the test program
   - Check for compilation errors

Problem: Startup Banner but No NMEA Data

Symptoms:

- Startup messages appear correctly
- "Waiting for NMEA data..." message shows
- But no $GP... sentences appear

Solutions:

1. **Check wire connections**
   - Verify yellow wire: GPS TX → ESP32 GPIO44 (RX)
   - Verify blue wire: ESP32 GPIO43 (TX) → GPS RX
   - Verify black wire: GND to GND
   - Wires firmly inserted?
2. **Check Waveshare jumpers**
   - Must be in **B-B position** (PI-LC29H mode)

- Not in A or C positions

3. **Verify GPS module powered**
    - Waveshare connected to Micro-USB?
    - Power LED on GPS module lit?
4. **Check for swapped TX/RX**
    - Common mistake: TX to TX, RX to RX (wrong!)
    - Correct: TX to RX, RX to TX (cross-over)
5. **Test GPS module separately**
    - Disconnect from ESP32
    - Connect GPS directly to PC via USB
    - Use terminal program to verify GPS sends data

Problem: GPS Shows No Fix (Empty Data)

Symptoms:

- NMEA sentences appear
- But latitude/longitude fields are empty
- Fix quality shows 0
- 0 satellites tracked

Solutions:

1. **GPS needs clear view of sky**
    - Move near window
    - Go outdoors
    - Remove any metal objects blocking antenna
    - Point antenna upward
2. **Wait for satellite acquisition**
    - Cold start can take 30-60 seconds
    - Warm start (after recent fix) takes 10-30 seconds
    - Watch PPS LED - should start blinking when fix acquired
3. **Check antenna**

- Integrated patch antenna should be on top of module
- Not covered or obstructed
- Module should be horizontal (antenna facing up)

4. **Check GPS module firmware**
    - Ensure it's LC29H(AA) version with OSNMA firmware
    - Check Waveshare product markings

Problem: Intermittent Data

Symptoms:

- Data appears, then stops
- Irregular updates
- Missing sentences

Solutions:

1. **Check power supply**
    - Use good quality USB cables
    - Try different USB ports
    - Check if PC goes to sleep/standby
2. **Check wire connections**
    - Loose jumper wires?
    - Wiggle wires to see if data stops/starts
    - Replace jumper wires if damaged
3. **Check for interference**
    - Move away from Wi-Fi routers
    - Move away from fluorescent lights
    - Reduce nearby electronic noise

Problem: Upload Fails

Symptoms:

- "Failed uploading: uploading error: exit status 2"
- "Could not open COM4"

- Upload hangs at "Connecting..."

Solutions:

1. **Close Serial Monitor**
    - Serial Monitor locks the COM port
    - Close it before uploading
2. **Try BOOT+RESET sequence**
    - Hold BOOT button on ESP32
    - Press and release RESET
    - Release BOOT
    - Immediately click Upload
3. **Run Arduino IDE as administrator**
    - Right-click Arduino IDE → Run as administrator
4. **Check if another program using COM port**
    - Close any terminal programs
    - Close other Arduino IDE windows
5. **Try different COM port**
    - Device Manager → Change COM port number
    - Select new port in Arduino IDE

Problem: PPS LED Not Blinking

Symptoms:

- GPS powered but PPS LED doesn't blink
- Or blinks irregularly

Solutions:

1. **GPS doesn't have satellite fix yet**
    - This is normal - wait 30-60 seconds
    - Move to better location with sky visibility
2. **Check GPS power**
    - Verify Micro-USB connected

- Power LED should be lit

3. **PPS LED may be disabled in firmware**
    - Some firmware versions have PPS disabled
    - Check with Waveshare documentation
    - GPS may still work even if PPS doesn't blink

---

14. FREQUENTLY ASKED QUESTIONS

Q1: Do I need an external antenna?

A: No. The Waveshare LC29H has an integrated patch antenna (the gray ceramic square on the board). This is sufficient for most applications.

Q2: How long does it take to get a GPS fix?

A:

- **Cold start** (first power-on or after long time off): 30-60 seconds
- **Warm start** (recent fix, ephemeris data valid): 10-30 seconds
- **Hot start** (just powered back on): 1-5 seconds

The board needs to download navigation data from satellites, which takes time.

Q3: Why does my GPS show a position far from my actual location?

A: This usually happens when:

- GPS doesn't have enough satellites (needs 4+ for 3D fix)
- GPS is using old/invalid ephemeris data
- Poor satellite geometry (all satellites in one direction)

Solution: Wait for more satellites to be acquired, move to better location.

Q4: Can I use this indoors?

A: GPS signals are very weak and don't penetrate buildings well. You may get:

- No fix at all
- Very slow fix acquisition

- Poor accuracy

Best results: Near window, outdoors, or on upper floors of buildings.

Q5: What is HDOP and why does it matter?

A: HDOP (Horizontal Dilution of Precision) indicates accuracy:

- **HDOP < 1.0:** Excellent (rare)
- **HDOP 1.0-2.0:** Good
- **HDOP 2.0-5.0:** Moderate
- **HDOP > 5.0:** Poor

Lower HDOP = better satellite geometry = more accurate position.

Q6: Why do I see different sentence types ($GP, $GA, $GL)?

A: The LC29H is a multi-GNSS receiver supporting:

- **$GP...:** GPS (USA)
- **$GA...:** Galileo (Europe)
- **$GL...:** GLONASS (Russia)
- **$GB...:** BeiDou (China)
- **$GN...:** Combined data from multiple systems

This gives you more satellites and better accuracy.

Q7: Can I send commands to the GPS module?

A: Yes! The TX pin (blue wire, GPIO43) allows you to send configuration commands. You can:

- Change update rate
- Enable/disable NMEA sentence types
- Configure OSNMA authentication
- Set baud rate

Commands use NMEA format or UBX protocol. See Quectel LC29H documentation for details.

Q8: What is the accuracy of the LC29H?

A:

- **Horizontal:** ~2.5 meters (CEP, open sky)
- **With SBAS/DGPS:** ~1 meter
- **With OSNMA (authenticated):** Same accuracy but verified authentic

Accuracy depends on satellite visibility, HDOP, and atmospheric conditions.

Q9: Why is the GPS data old (wrong date/time)?

A: GPS has not downloaded valid almanac/ephemeris data yet. The date shown may be:

- GPS epoch start (1980)
- Date of last valid fix

Solution: Wait for full satellite lock with valid navigation data.

Q10: Can I power the GPS from the ESP32 3.3V pin?

A: Technically yes, but not recommended:

- GPS can draw significant current during acquisition (peak ~100mA)
- ESP32 3.3V pin may not supply enough current
- Could cause brownouts or instability

Best practice: Use separate USB power as shown in this lab.

Q11: What does "AA" mean in LC29H(AA)?

A: The (AA) designation indicates this module has:

- OSNMA-capable firmware
- Galileo authentication support
- Special firmware version for cryptographic validation

Only LC29H(AA) can validate Galileo OSNMA signatures. Other versions (LC29H, LC29HBS, etc.) cannot.

Q12: Why are there so many NMEA sentences?

A: Different sentences provide different information:

- **$GPGGA:** Position and fix data
- **$GPRMC:** Time, date, position, speed
- **$GPGSV:** Satellite details
- **$GPGSA:** Which satellites are used

Most applications use just $GPGGA or $GPRMC. Others provide additional context and diagnostics.

---

15. NEXT STEPS

What You've Accomplished

Congratulations! You have successfully:

✓ Connected a GPS receiver to a microcontroller
✓ Configured UART serial communication
✓ Received real satellite navigation data
✓ Displayed NMEA sentences in real-time
✓ Understood basic GPS operation

This is the foundation for more advanced GPS applications!

Enhancement Ideas

Level 1 - Parse NMEA Data:

- Extract latitude and longitude from $GPGGA
- Display position in decimal degrees
- Show altitude and number of satellites
- Calculate distance traveled

Level 2 - Log GPS Data:

- Store NMEA sentences to SD card
- Create GPX track files
- Time-stamp all data
- Build GPS logger

Level 3 - OSNMA Authentication:

- Receive OSNMA navigation messages
- Verify cryptographic signatures

- Validate satellite authenticity
- Detect spoofing attempts

Level 4 - Mapping and Visualization:

- Plot position on map
- Calculate speed and heading
- Create real-time tracking display
- Geofencing and waypoint navigation

Additional Resources

NMEA Protocol:

- NMEA 0183 Standard Documentation
- Online NMEA sentence decoders

GPS Theory:

- How GPS works (trilateration)
- Satellite orbits and geometry
- Atmospheric error sources

LC29H Module:

- Quectel LC29H datasheet
- Waveshare product documentation
- OSNMA user guide

Arduino ESP32:

- ESP32 UART advanced features
- DMA for high-speed serial
- Multiple serial port usage

BIP26 Programme Continuation

This lab is **Module 1** of the BIP26 SENSATE-X programme. Upcoming modules:

Module 2: OSNMA Message Structure
Module 3: Cryptographic Signature Verification
Module 4: Galileo Authentication Service
Module 5: Spoofing Detection
Module 6: Final Project

Assessment

To demonstrate competency, you should be able to:

1. Explain the difference between TX and RX pins
2. Configure UART baud rate and format (8N1)
3. Identify correct jumper settings on GPS module
4. Wire GPS to microcontroller without errors
5. Upload and run Arduino test program
6. Interpret NMEA sentence output
7. Troubleshoot common connection problems
8. Explain basic GPS operation

Lab Report Requirements

Your lab report should include:

1. **Cover page:** Title, name, date, course
2. **Objectives:** What you aimed to achieve
3. **Procedure:** Steps you followed (brief summary)
4. **Results:** Screenshot of Serial Monitor showing NMEA data
5. **Analysis:** One example NMEA sentence fully explained
6. **Challenges:** Any problems you encountered and how you solved them
7. **Conclusion:** What you learned
8. **Appendix:** Your Arduino code (if modified from template)

Submission: Upload to Brightspace by [deadline]

ACKNOWLEDGMENTS

Hardware:

- TenStar TS-ESP32-S3 development board
- Waveshare LC29H(AA) GNSS module with Quectel chip

Open Source Tools:

- Arduino IDE and ESP32 board support

- Espressif ESP-IDF framework
- NMEA 0183 protocol standard

Contributors:

- BIP26 SENSATE-X programme team
- Partner institutions: H-DA, UTCluj, UPV, FH Wien, UniCas
- TU Dublin Electrical & Electronic Engineering

DOCUMENT VERSION HISTORY

Version	Date	Changes	Author
1.0	April 2026	Initial release	Gerard Stockil

END OF LABORATORY MANUAL

This lab manual is part of the BIP26 SENSATE-X Blended Intensive Programme funded by Erasmus+.

For questions or clarification, contact:
Gerard Stockil
gerard.stockil@tudublin.ie
TU Dublin Tallaght Campus

That's the complete content! Copy everything above into Microsoft Word, add your photos where indicated, and you'll have a complete professional lab manual ready for your students!

Would you like me to adjust anything or add more sections?

Perfect! I'll add the power-up safety section to the lab manual. Here's the complete updated section to insert into your DOCX:
